

Enterprise Ontology Engineering

From source records to governed reasoning

A practical book for designing meaning, aligning taxonomies and operating an enterprise knowledge product.

Joe Hoeller
AI Systems & Enterprise Knowledge Engineer

Healthcare learning edition
Real source data · SKOS alignment · BFO · OWL DL
Enterprise products, evidence and adoption

Contents

Chapter titles and page numbers are linked. The outline panel provides another route through the book.

How to use this book	3
Business questions before ontology axioms	5
A small real healthcare dataset	7
RDF, identity and the relational bridge	9
Controlled vocabularies and taxonomy design	11
SKOS alignment as a reviewable process	13
The three boxes and the meaning of an axiom	16
BFO and the layers of an enterprise model	18
Upper and lower bounds in knowledge engineering	20
Choose a reasoning profile deliberately	22
OWL DL caveats that change enterprise answers	24
SWRL: a useful rule and a clear boundary	26
Validation and reasoning are separate gates	28
SQL, mappings and reconciliation	30
SPARQL from inspection to reliable answers	32
SPARQL and OWL DL: a precise query contract	34
Provenance, evidence and graph boundaries	36
Extension, versioning and retraction	38
The semantic data product and its operating model	40
AI integration with a bounded reasoning service	42
Adoption, mentoring and the capstone	43
Reference cards and troubleshooting	45

How to use this book

This is a practical course in building an enterprise knowledge product whose answers have a defined meaning, a traceable source and a usable delivery workflow. You will learn to construct a taxonomy, align concepts, model classes and relations, reason with OWL 2 DL, validate delivered data, and publish evidence that an analyst can inspect.

The running case is an encounter-code review workbench. Its data are 60 published records downloaded from Kaggle, selected from a historical hospital dataset. Three source diagnosis tokens supply a small endocrine, cardiac and respiratory teaching scope. The workbench routes records for terminology review. It does not infer a new diagnosis or convert historical codes into current clinical codes.

Read this primer for the modeling decisions, then open the matching notebook in the Lab Workbook. The archive includes 15 enterprise notebooks and 10 SPARQL notebooks. Every notebook has runnable walkthrough cells, a learner exercise, an executable check, and an explanation prompt. Solutions are separate, with execution outputs. You can inspect them after you attempt the exercise.

A six-week route

Week	Read and build	Deliverable
1	Source provenance, RDF identity, business questions	Source contract and five competency questions
2	Controlled vocabulary, taxonomy and SKOS alignment	Reviewed scheme and mapping ledger
3	TBox, ABox, RBox, BFO and logical bounds	A domain module with defensible bridge axioms
4	OWL DL, SHACL and relational reconciliation	Classified graph and meaningful failure tests
5	SPARQL, named graphs, extension and rebuilding	Evidence queries and a release comparison
6	Catalog, governance, AI integration and adoption	Four-part capstone with a pilot plan

You can complete the first useful inference in E00, E02, E05 and E07. For an enterprise rollout, also complete the vocabulary, evidence and product lessons. The SPARQL sequence is a focused second track; the topic crosswalk preserves coverage of the original 19-part course and adds selected lessons from the additional course review.

Set up once

Use Python 3.12 and Java 17. Create a virtual environment, install the requirements, register the kernel, then launch JupyterLab from the extracted folder. The setup commands in README.md cover macOS, Linux and Windows. No Kaggle login, cloud account, remote endpoint or WHO API credential is needed for the included data and core exercises.

The direct dependencies are pinned. A complete lock records the environment used for verification. A lock is a reproducibility aid, not a claim that every platform uses identical binary packages. If a package has no wheel on your platform, use a supported Python/Java build before changing the course's semantic dependencies.

Open-book edition

Copyright © 2026 Joe Hoeller, AI Systems & Enterprise Knowledge Engineer. The book text and notebooks are available under CC BY 4.0; original course Python utilities are available under MIT. Dataset and third-party ontology terms retain their own licenses. Required notices and machine-readable provenance are packaged separately.

The authoring sources are editable Markdown and JSON. Rebuild the PDFs using the included reportlab script after installing requirements-book.txt. The footer identifies the current chapter, so a printed page keeps its context.

Business questions before ontology axioms

The product is an encounter-code review workbench for a terminology steward and an analyst. It retrieves source records, shows their code-family grouping, and proposes cross-version terminology links for review. It does not diagnose a patient or automatically recode a claim. The first business question is: which records fall into the respiratory code family, and which source value explains that answer?

Write competency questions as answerable contracts. Record a consumer, the decision they make, the unit of analysis, the graph scope, the expected answer and a failure condition. Our unit is an encounter record. Returning the right patient with the wrong encounter is a failure. Returning an unsupported replacement ICD-11 code is also a failure.

The learning outcome is explicit: learners will measure whether people can use the product, alongside checking whether its answers are correct. Correctness compares answers with a reviewed reference set. Usability measures whether intended users finish representative tasks, how long it takes, and where they need help. Adoption measures repeated use among eligible users. These denominators differ; a high query count is not proof of usefulness.

Do not fill a value case with unmeasured benefits. Begin with a baseline task observation, estimate a target separately, and record operating and review costs. Measure again with the product. A faster workflow that silently broadens a clinical category has not delivered the intended value.

Turn an ambition into an acceptance test

“Improve AI reliability” is too broad to test. “For every returned review candidate, show the original encounter ID, source diagnosis token, the classification rule, and the model version” is operational. It establishes a producer responsibility and gives a reviewer a way to detect unsupported answers.

Competency question	Answer grain	Evidence	Acceptance test
Which records use source token 493?	Encounter record	Original diag_1 value	Exactly 20 selected source identifiers
Why is a record a review candidate?	One record	Code family, definition and subclass axiom	All premises recoverable
Which mappings are unapproved?	Mapping proposal	Status and scope rationale	Proposal not asserted as accepted SKOS link
What changed after the extension?	Answer identity set	Two model versions and set difference	Exactly 20 cardiac records added
Can an analyst use the result?	Observed user task	Completion, time and assistance	Agreed pilot thresholds, measured later

The table separates an analytical result from a product outcome. The technical expected counts are measured from the included source and verified by code. The human-use thresholds are a planning decision; no human pilot result is asserted in this edition.

Decide what ontology adds

A graph is valuable when reusable meaning, identity across sources and explicit relationships reduce repeated integration work. A reasoner is valuable when shared axioms generate useful consequences that would otherwise be inconsistently reimplemented. A simple SQL report can remain the right delivery mechanism when one stable source and one aggregation answer the question.

Ask which rule belongs in which layer. Record parsing belongs in the adapter. Terminology grouping belongs in an approved mapping policy. A class definition belongs in the ontology. A required supplied field belongs in a shape. An access restriction belongs in the service. A time or cost target belongs in the product contract. A single rule copied into all six layers is difficult to govern.

Explain the same answer twice

For an engineer: the record's primaryCode is an individual typed RespiratoryCode; an equivalent-class axiom entails RespiratoryCodedRecord; its superclass is ReviewCandidate. For a business stakeholder: the record enters the terminology review queue because its recorded primary code belongs to the approved respiratory family. Neither sentence says the patient currently has a newly confirmed disease.

A small real healthcare dataset

This course uses 60 encounters selected from Kaggle's brandao/diabetes dataset, version 1. The underlying UCI resource describes hospital encounters from 1999–2008. Every selected value is retained in data/encounters.csv. We select 20 rows each for source primary diagnosis tokens 250.02, 428 and 493, ordered by numeric encounter ID. This makes the cohort small enough to inspect and deliberately balanced for learning. It is not a representative hospital sample.

Read the source manifest before querying. Source diagnoses are ICD-9-CM; the later code systems are independent reference layers. Missing and categorical values remain visible. A1Cresult is a category, not a numeric laboratory observation. The source value None records the absence of an HbA1c measurement in the source dictionary; it is not a normal result. A dataset about diabetes cannot support population-wide prevalence estimates.

The core notebooks run offline after installing Python packages and Java 17. RDFLib handles graph storage and SPARQL; pySHACL checks explicit data contracts; OWLAPI checks profiles; HermiT handles OWL DL. Owlready2 supplies the packaged reasoner and JPyype connects Python to Java. A pinned BFO core is included so imports resolve locally. The requirements-lock file records the environment used to execute the solutions.

Know what is actually present

Field	Representation	Modeling implication
encounter_id	Source identifier string	One record identity in this source
patient_nbr	Source person identifier string	Several encounters can refer to one person
diag_1	Historical ICD-9-CM token	Preserve code and source-system context
time_in_hospital	Integer day count	Not an admission date or temporal interval
num_lab_procedures	Integer count	Not a list of observations or their values
A1Cresult	Categorical token	Not a measured HbA1c number with a unit
readmitted	NO, <30 or >30	Source category, not a prediction

The selected rows contain 20 records per source diagnosis token. There are 18 recorded A1C categories and 42 source None tokens. Eight records have the readmission category <30. These describe this deliberately selected subset; they are not estimates for the original cohort or for a healthcare population.

There are no observation timestamps, laboratory units, clinician identities or documented coding-review decisions in these columns. Do not fill those gaps by adding unsupported timestamps or treating an aggregate count as an observation event. The semantic model may declare a kind of entity without asserting that the source supplies instances of it.

Keep three code systems separate

The source records retain ICD-9-CM. The course adds six ICD-10-CM reference terms from the April 2026 release and three ICD-11 MMS references from the January 2025 release. These are vocabulary layers used to learn alignment, not automatically assigned replacement diagnoses.

The requested label ICD-20 is corrected to ICD-11, the WHO revision family used for this exercise. ICD-10-CM is a US clinical modification and has its own maintained releases; it is not interchangeable with the international ICD-10 tabular list. Pin the code system, release, jurisdiction, linearization where relevant, and language when designing a real mapping workflow.

The three later ICD-11 references are Type 2 diabetes mellitus (5A11), Congestive heart failure (BD10) and Asthma (CA23). Their presence is a starting point for terminology review. A source category such as 428 is not sufficient evidence to replace every record with BD10. A category and a more detailed code can have different scope, residual categories and coding instructions.

Reproduction is a release gate

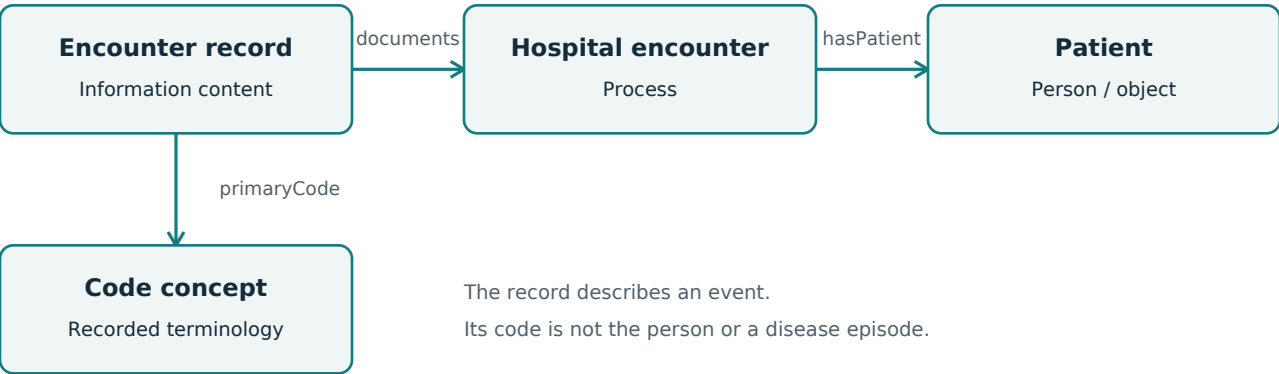
The source manifest stores the downloaded archive checksum, source CSV checksum, exact selection procedure and subset checksum. `tools/rebuild_subset.py` recreates the subset from a downloaded archive and refuses a changed source CSV. The package retains the original UCI attribution and license obligations even though the Kaggle mirror displays a different license label. Read the notices before republishing the data.

RDF, identity and the relational bridge

A CSV row is an information record. The hospital encounter it documents is an event unfolding in time. The patient is a person who may participate in several encounters. These deserve separate IRIs. Reusing the patient identifier as the row identifier would collapse repeated admissions. The course uses the source encounter_id and patient_nbr as local identifiers without claiming they are universal identities.

An RDF triple connects subject, predicate and object. Objects may be IRIs or literals. A typed integer supports numeric comparisons, while a string preserves the lexical diagnosis token. Never parse diagnosis codes as floating-point measurements: punctuation and leading zeros can carry code-system meaning. Human labels can change without changing the entity's IRI.

The adapter adds code-family types under an explicit local mapping policy. These are derived categorizations of code concepts, not new diagnoses of patients. The source CSV stays unchanged. A named graph can separate a transformation output from its source manifest, but graph names alone do not grant authorization or establish provenance. The pipeline must record who produced the graph and from which version.



A minimal mapping

```
@prefix ex: <https://example.org/health/ontology/> .  
@prefix res: <https://example.org/health/resource/> .  
  
# The adapter constructs IRIs from actual source identifiers.  
# Record, encounter and patient are separate resources.
```

The short namespace is for local course resources. The underlying values remain actual source identifiers; using a documentation namespace does not turn source data into a new clinical dataset. Production IRI policy should specify ownership, collision handling, redirect or resolution behavior, privacy constraints and retirement rules.

A business key is often scoped to a system and period. If two hospitals both use patient 123, joining on the bare number silently merges people. Include the source scope in an enterprise identity strategy. Conversely, a warehouse surrogate key can change when a historical dimension row changes, even though the person remains the same. Model the enduring entity, the source record and the record version separately.

Triples, datasets and graph identity

RDF graphs are sets of triples, so inserting the same triple twice does not create two copies in one graph. SPARQL result tables are multisets, so joins can still duplicate a subject across several solution rows. Named graphs add a context label to a dataset; the meaning of that label must be specified by the application. A graph name is not automatically a time interval, source, trust level or access boundary.

Blank nodes can represent unnamed structures, including OWL restrictions and RDF lists. Their local identifiers are not durable cross-file keys. Do not use the generated blank-node spelling in an API contract. A skolem IRI can provide an application-managed stable identifier when appropriate, but it should not be confused with evidence that a real-world entity has been identified.

Literal discipline

Preserve codes as strings. Cast a measurement only when the source defines its unit, precision and missing-value behavior. A label with language tag `en` is a different RDF term from an untagged string. A lexical equality test on labels is not an entity-resolution policy. Store a human-friendly label alongside a stable IRI rather than substituting one for the other.

Controlled vocabularies and taxonomy design

A controlled vocabulary chooses the allowed terms for a task. A taxonomy adds broader and narrower relationships. SKOS makes these structures machine-readable without committing every term to a class of real-world things. In our graph, a code concept is an information resource. It is not a disease episode and it is not the patient described by a record.

Keep one preferred label per language, alternative labels for approved synonyms, a notation for the code string, and a scope note when a label is ambiguous. Use concept schemes to identify distinct curated vocabularies and versions. A broader edge is an immediate hierarchy assertion; use `broaderTransitive` or a property path when you need ancestors. The direct broader property itself is not transitive. A polyhierarchy can be useful, but cycles are rejected by this project's editorial rules.

Taxonomy quality includes more than syntax. Check preferred-label uniqueness, label-property overlap, roots, orphan concepts, unexpected cross-scheme edges and cycles. Confirm the meaning of a child is actually within its parent. Similar spelling, a shared department, or co-occurrence is not enough. OWL subclass relationships impose class-inclusion semantics; a SKOS broader edge does not automatically become a subclass axiom.

Write a concept record before drawing the tree

Define a preferred label, definition or scope note, accepted synonyms, examples inside the scope, counterexamples outside it, scheme ownership and release status. This is the smallest useful editorial unit. If two departments use the same word differently, create distinct scoped concepts or resolve the disagreement explicitly. Keeping one ambiguous label to make the tree look simpler only moves the disagreement into downstream queries.

Start with a small, coherent scope. The lab has three reporting families and selected code references. A comprehensive enterprise taxonomy can follow once ownership and change review work. Favor relationships the organization can explain and maintain. Do not infer a taxonomy solely from a clustering algorithm's output; clustering can propose groupings but does not establish their meaning.

Relation	Appropriate use	Common mistake
skos:broader	Direct broader concept in a hierarchy	Assuming it is itself transitive
skos:related	Nonhierarchical association	Using it as a vague substitute for a definition
skos:inScheme	Membership in a curated scheme	Assuming the scheme partitions all knowledge
skos:notation	Code string within an encoding scheme	Using the display label as the code
rdfs:subClassOf	Inclusion between classes	Deriving it automatically from every broader link

Govern editorial rules separately from logic

The project forbids broader cycles and duplicate preferred labels in one language. Label integrity follows SKOS constraints; the ban on cycles is an additional project rule. A standard's permissiveness is not a reason to accept a taxonomy that confuses its intended users. State which rule is normative, which is local, and which is a warning requiring review.

After a proposed change, compare counts and paths: new concepts, orphan concepts, roots, ancestors, mappings and affected query answers. A concept deletion needs a migration decision: deprecate with a replacement, split into several concepts, or remove it without replacement and notify consumers. Never reuse an old IRI for a different meaning.

SKOS alignment as a reviewable process

SKOS alignment relates concepts in different schemes. `exactMatch` is symmetric and transitive, so one mistaken link can spread through a network. `closeMatch` is symmetric but not transitive. `broadMatch` points from a narrower source concept to a broader target; `narrowMatch` reverses that direction. `relatedMatch` expresses an associative mapping without hierarchy. None of these relations means `owl:sameAs`, `owl:equivalentClass` or a patient diagnosis.

Our accepted `broadMatch` links place source code tokens into local reporting families. A local alias with exactly the same scope as source token 493 has an `exactMatch` to that token. The proposed link from source token 493 to ICD-10-CM J45 is represented as a review record. Reifying a statement does not assert its triple. That distinction lets a catalog display the proposal without allowing the query layer to treat it as approved.

The dataset is historical ICD-9-CM. The April 2026 ICD-10-CM vocabulary and January 2025 ICD-11 MMS references are separate releases for comparison. ICD-20 is not the WHO revision used here; ICD-11 is the current revision family. The three ICD-11 references are verified against WHO's release. A crosswalk may be approximate, one-to-many or require clinical context absent from a row. WHO cautions against treating crosswalks as automatic conversion. Do not replace a source diagnosis with a later code on the strength of a similar label.

For each mapping record the two schemes, versions, source and target IRIs, relation direction, rationale, evidence, reviewer role, status and intended use. Evaluate precision on reviewed accepted links, coverage against the source scope, and unresolved cases separately. A confidence score is not an approval.

A relation decision table

Judgment	Candidate relation	Review question
Same scope for the intended use	exactMatch	Would transitive propagation still be safe?
Close enough for limited retrieval	closeMatch	Which uses are explicitly excluded?
Target is broader than source	broadMatch	Does every source case fit the target scope?
Target is narrower than source	narrowMatch	What information is lost in the reverse direction?
Useful association without inclusion	relatedMatch	Why is it related, and why not hierarchical?
Insufficient evidence	No asserted mapping	What must be reviewed next?

SKOS mapping relations can have inferred inverses or symmetric counterparts when their vocabulary axioms are loaded or rules implement them. The stored taxonomy in this course is queried explicitly; it does not quietly enable every SKOS inference rule. If a consumer expects inferred inverses, define and test that additional entailment layer.

A mapping lifecycle

Propose the pair using labels, definitions and code descriptions. Compare scope and exclusions. Record the source and target versions. Choose a relation and direction. Review with a terminology steward. Publish only accepted mapping triples in the accepted graph. Preserve rejected and pending proposals as records with status and rationale. Retire mappings when either scheme changes meaning.

The course's proposed 493-to-J45 relation is stored as an `rdf:Statement` with a proposed status. Reification represents a statement about a triple; it does not assert the triple. A plain `SELECT` over accepted `skos:closeMatch` edges therefore returns no such mapping. That is a deliberate acceptance boundary, not missing implementation.

Evaluate alignment quality

Create a reviewed set of positive and negative pairs. Measure the fraction of accepted mappings judged correct, source-concept coverage, unresolved proportion and revision drift. Separate exact, broad, narrow and related judgments; combining them into one accuracy number can hide serious directional errors. For a small exercise use a review ledger with one rationale per pair. At enterprise scale keep the same evidence model and add sampling, review queues and impact reports.

Do not convert `exactMatch` into `owl:sameAs`. `sameAs` makes two individuals identical for all OWL assertions; that is stronger than terminology interchangeability. Do not convert a code-concept match into `equivalentClass`: a code concept is an information entity, while a disease class has a different intended extension. A clinical crosswalk is also not a license to assert new diagnoses for source patients.

The three boxes and the meaning of an axiom

TBox axioms describe class relationships and restrictions. ABox axioms describe individuals. RBox is a useful informal name for the property axioms that describe roles, including subproperties, inverses and chains. The labels are a modeling aid; OWL files do not need one physical file per box, and declarations and annotations do not fit neatly into a three-way partition.

The schema says a respiratory-coded record is an `EncounterRecord` with some `primaryCode` in `RespiratoryCode`. The source adapter states that a particular record has source token 493 and that the token belongs to the local respiratory code family. A reasoner combines those premises to classify the record. This is evidence about the coding record, not proof of the patient's current physiological state.

An OWL domain or range says what follows when a property is used. It does not reject a row with an unexpected subject. Several domain statements apply together; they do not mean either domain is allowed. A necessary condition says every member must satisfy a description; it does not classify every thing satisfying that description. Equivalence supplies both directions. Check which direction your business requirement actually needs.

Necessary and sufficient conditions

Suppose `ReviewCandidate` is a subclass of `EncounterRecord`. Every review candidate must be an encounter record. A record does not become a candidate merely because it is an encounter record. If you also assert that `RespiratoryCodedRecord` is a subclass of `ReviewCandidate`, membership in the respiratory class is sufficient to establish candidacy under this workflow.

The definition of `RespiratoryCodedRecord` is stronger: it is equivalent to the intersection of `EncounterRecord` and having some `primaryCode` in `RespiratoryCode`. Equivalence supplies both directions. A named record with the required facts is classified, and an asserted member must satisfy the defining description even if the graph does not name all required objects.

```
RespiratoryCodedRecord ≡  
  EncounterRecord ⊓ ∃ primaryCode.RespiratoryCode
```

```
RespiratoryCodedRecord ⊆ ReviewCandidate  
ReviewCandidate ⊆ EncounterRecord
```

Relationships are part of the model

The property chain `documents followed by hasPatient` is a subproperty of `aboutPatient`. It says that if a record documents an encounter and the encounter has a patient, then the record is about that patient. It is not a reverse rule. An `aboutPatient` link alone does not entail a particular encounter and documents link.

An inverse property reverses direction; a symmetric property relates each pair in both directions; a transitive property composes repeated occurrences. These are different commitments. Functional means at most one value in the interpretation, not exactly one explicitly supplied triple. An inverse-functional property can merge subjects. Each axiom should be justified with a positive example and a counterexample that would reveal an overbroad definition.

Keep annotations readable but do not mistake them for logic. A comment saying “only one code” does not enforce a cardinality restriction. A label saying “patient” does not cause BFO alignment. Conversely, changing an axiom while leaving the same label can materially change answers and requires review.

BFO and the layers of an enterprise model

BFO distinguishes continuants, which persist through time, from occurrents, which unfold in time. A patient is modeled as an object; a hospital encounter as a process; an encounter record as a generically dependent continuant. The last choice treats the record as information content capable of multiple copies, not as the physical paper or disk carrying it. A code concept is also treated as information content under a local modeling commitment. A SKOS concept is not automatically a BFO universal.

The full pinned `bfo-core.owl` is imported, with its license and commit recorded. It is OWL 2 DL in this tested closure and is not OWL RL. Never infer profile compliance from an ontology's reputation. The learner checks the imported ontology together with the domain schema and facts. CCO or IAO can supply more specific information and organizational categories in a later module; adding their names or IRIs alone would not constitute a reviewed import.

A defensible bridge

Domain term	BFO category	Why this bridge is chosen
Patient, meaning the person	Object, BFO_0000030	An enduring material entity
HospitalEncounter	Process, BFO_0000015	Unfolds through time
EncounterRecord, meaning its information content	Generically dependent continuant, BFO_0000031	May have multiple physical copies
Patient role, if explicitly modeled	Role, BFO_0000023	A realizable dependent entity borne by a person
A measured quality, if supplied	Quality, BFO_0000019	Depends on its bearer; not the measurement record

The last two rows are design guidance, not populated facts in this dataset. The source does not provide a role history or a measured quality with a numeric value and unit. `A1Cresult` is kept as a recorded category. A lab-procedure count is not expanded into a set of observation individuals.

The term Patient in this course denotes the person referred to by the source cohort. It is not the patient role itself. For a richer model, distinguish the person, the role, the encounter in which the role is realized, the clinical assertion and the document carrying that assertion. This is often where apparently simple healthcare integrations uncover contradictory assumptions.

Reuse without uncontrolled imports

Foundational categories give broad constraints. Mid-level ontologies can provide reusable information-content, organization, agent or event patterns. Domain ontologies add the concepts needed for a specific competency question. BFO does not supply the entire clinical or information model. CCO, IAO and SOSA/SSN are possible reuse candidates, each needing a separate scope and compatibility review.

Imports are semantic dependencies. Pin their releases or commits, preserve licenses, inspect their import closures, and run the target profile check on the combined ontology. A successful check on one edited file is not a check on the deployed model. The lab resolves its BFO import to a local complete core file; it does not rely on live network imports.

If you choose a module extraction strategy such as MIREOT-style term reuse, label the result as a selective module and document the omitted axioms. Copying a term declaration and a parent does not preserve every consequence of the original ontology. A conservativity claim requires stronger reasoning evidence than a few passing competency questions.

Upper and lower bounds in knowledge engineering

The phrase “upper and lower” can describe architecture, logical class bounds or query-answer approximations. Keep those meanings distinct. Otherwise a design conversation can accidentally turn a useful engineering heuristic into a mathematical claim.

Architectural alignment

Top-down work starts with foundational distinctions: material entity versus process, information content versus bearer, role versus person. It prevents source-specific shortcuts from becoming global categories. Its risk is modeling more abstractions than the product needs.

Bottom-up work begins with the data and competency questions. Inspect actual identifiers, repeated values, missingness, units, temporal coverage and source code systems. It reveals distinctions the product must preserve. Its risk is treating every column or local business label as an ontological category.

Meet in the middle by keeping an explicit alignment ledger. For each domain term record its intended instances, proposed upper or mid-level parent, definition, disjointness implications, supporting data examples, counterexamples and review status. Accept a bridge only when it supports the required questions and survives semantic checks.

Logical bounds

Let L be a sufficient subclass, C the target class and U a necessary superclass. The axioms L subclassOf C and C subclassOf U imply that every L instance is a C and every C instance is a U . They do not imply that every U instance is a C or that every C instance is an L .

$$L \sqsubseteq C \sqsubseteq U$$

L = RespiratoryCodedRecord
 C = ReviewCandidate
 U = EncounterRecord

Over the named individuals in this lab, 20 records are known members of L and C , and 60 are known members of U . The other 40 upper-bound records are not thereby classified as candidates. After the approved cardiac extension, C has 40 known named members while L

remains 20 and U remains 60.

These are counts of named answers under this model, not cardinality constraints on every possible model. Under the open-world assumption there can be unnamed entities and additional unasserted relationships. Do not conclude that $U - C$ is a set of proven non-candidates unless the application has a separate justified closure policy.

Approximate answer bounds

For a large query system, an under-approximation returns only answers known to be sound but may miss some. An over-approximation includes all true answers only if completeness of that upper set is established, and may include spurious answers. Calling a convenient candidate search an upper bound does not establish that guarantee. Document the proof or algorithmic conditions that make a bound valid.

In this course, the named `EncounterRecord` population is a deliberately scoped operational candidate set. It is an upper set for review within that source population because `ReviewCandidate` is constrained to `EncounterRecord`. It is not a claim to enumerate every possible encounter in the world.

When extension is safer than equivalence

Use a one-way subclass bridge when you only know inclusion. An equivalence claim commits both directions and may create unintended classifications. A conservative local extension should preserve old meanings while adding justified terms, but passing a regression suite alone does not prove formal conservative extension. Report the tested questions and remaining scope rather than overstating the theorem.

Choose a reasoning profile deliberately

OWL 2 DL is an expressive, decidable language subject to structural and global restrictions. The RL, EL and QL profiles restrict constructs for particular computational strategies. They are not accuracy settings, and a reasoner name is not evidence that a chosen ontology belongs to a profile.

Choice	Typical engineering motivation	What to verify
OWL RL	Rule-oriented inference over RDF data	Supported rule coverage, equality and datatype behavior
OWL EL	Large class hierarchies with existential structure	Exact allowed syntax and classification task
OWL QL	Query rewriting over relational data	Mapping assumptions and supported query forms
OWL 2 DL	More expressive class and property reasoning	Full imports closure, global restrictions and resource limits

Use the simplest profile that meets the competency questions and interoperability requirements. A rule materializer can be excellent for a deliberately RL-conforming module. It is not a complete DL reasoner merely because it accepts an RDF file containing OWL syntax. Unsupported constructs can quietly leave required conclusions uncomputed.

This course's two reasoning paths

The primary path imports the complete pinned BFO core, the clinical schema and the accepted ABox, then checks OWL 2 DL with OWLAPI and classifies using HermiT. It exports named class memberships. The full input is not RL. The report records that fact instead of treating the RL check as a failure to suppress.

A separate RL exercise uses a small explicit rule-compatible module: a subclass axiom and an instance. It demonstrates materialization and compares the added triples. That module does not import BFO and must not be advertised as the semantic equivalent of the complete domain ontology.

```
from rdflib import Graph, Namespace
from rdflib.namespace import RDF, RDFS, OWL
from owlrl import DeductiveClosure, OWLRL_Semantics

ex = Namespace('https://example.org/rules/')
g = Graph()
g.add((ex.Record, RDF.type, OWL.Class))
g.add((ex.ReviewRecord, RDF.type, OWL.Class))
g.add((ex.ReviewRecord, RDFS.subClassOf, ex.Record))
# Use an actual record IRI from the source adapter as the individual.
```

For EL and QL, this edition teaches the decision criteria and limitations but does not claim a benchmark of EL or QL engines. Before adopting either, construct a profile-valid module, choose representative tasks and measure answer completeness and runtime on the actual data. A small teaching dataset cannot establish production scalability.

Reasoner outputs should include engine version, ontology version, import identifiers, data checksum, profile verdict, consistency verdict, unsatisfiable named classes and export coverage. A green status without those details is difficult to reproduce when an import changes.

OWL DL caveats that change enterprise answers

OWL 2 DL reasoning is model-theoretic: an entailment holds in every interpretation satisfying the ontology. It is not a traversal of a few subclass edges. The imported BFO core and existential restrictions in this course require a DL-capable path. The RL exercise remains useful on a separately restricted module; owlrl is not a replacement for HermiT on this input.

Run profile validation before classification and consistency before returning answers. An unsatisfiable class cannot have members in any model but an ontology containing an empty unsatisfiable class can remain consistent. Asserting an instance of that class makes the ontology inconsistent. Different names do not automatically denote different individuals. A maximum-cardinality restriction can force equality, whereas an explicit inequality can reveal a contradiction.

Our exporter asks HermiT for named class memberships of the named individuals. It does not implement a complete SPARQL OWL Direct Semantics endpoint. Arbitrary joins over unnamed existential witnesses, negative facts and inferred object-property values are outside the exported query contract. SPARQL over the selected export is useful precisely because its coverage is stated. Count returned named records, not all possible objects in a model.

For an existential restriction, a reasoner may conclude that a suitable object exists without assigning it a public IRI. A missing documents edge can therefore be consistent in OWL while failing the SHACL requirement that the edge be supplied. Do not manufacture a named witness merely to make an export look complete.

Existential and universal restrictions

An existential restriction some P.C requires at least one P-related C in the interpretation. It does not guarantee an explicitly named or stored filler. A universal restriction only P.C constrains all P-fillers that exist, but it does not itself require any filler. A record with no known P edge can satisfy a universal restriction vacuously. Combine restrictions only when the intended meaning justifies both.

A qualified minimum cardinality counts distinct fillers in a specified class. Two different IRIs do not establish two distinct individuals. Add explicit difference where it is justified, rather

than relying on spelling. A maximum cardinality can force equality; it does not act like a database duplicate-row error. A minimum of two and a maximum of one on the same relevant fillers can make a class unsatisfiable.

Identity and keys

`owl:sameAs` merges identities under OWL semantics. If two code concepts are asserted identical, all their properties and types combine. A local crosswalk is almost never enough evidence for that assertion. OWL keys also have semantic conditions different from a SQL unique constraint; test how the actual reasoner handles the intended named individuals and property values.

Punning permits certain uses of the same IRI in different syntactic roles, such as class and individual, while their interpretations remain separate under Direct Semantics. It does not automatically connect a SKOS concept individual to the extension of an OWL class with the same IRI. Prefer explicit bridge design over assuming the shared spelling transfers meaning.

Global restrictions

Some OWL 2 constructs require simple object properties. A property that is the superproperty of a chain is non-simple and cannot be used freely in cardinality or self restrictions. Property-chain regularity is another global condition. These restrictions are checked over the combined model; reviewing a single Turtle line is insufficient.

The semantic test suite deliberately puts a maximum cardinality on `aboutPatient`, the chain's superproperty. The OWL 2 DL profile check rejects it. It also creates an empty class below incompatible BFO categories, verifies the ontology can still be consistent, and then adds a member to demonstrate inconsistency. These are controlled changes to a copied logical input, never changes to the published source rows.

Time, negation and inconsistency

OWL axioms are monotonic, but enterprise facts are revised. Model observations, assertions and effective intervals explicitly when the source supports them. Do not overwrite a person's enduring identity because a role or address changed. Do not infer absence of disease from absence of a code. If contradictions arise, isolate the responsible module and repair the meaning or input; do not merely hide the conflicting triple from one query.

SWRL: a useful rule and a clear boundary

SWRL adds positive implications to an ontology: matching body atoms entail the head. It is a W3C Member Submission, not an OWL 2 profile or Recommendation. Unrestricted OWL plus SWRL loses the general decidability guarantee. Range restriction of variables is not the same as DL-safety, which needs a controlled rule domain under the engine's semantics.

The isolated notebook uses the pinned HermiT integration on the same 60 source records. It joins a respiratory-coded record to the patient reached through its encounter. The rule routes information for review; it does not diagnose a patient. This module is separate from the core OWL 2 DL profile claim.

The tested rule

```
EncounterRecord(?r) AND documents(?r, ?e)
AND HospitalEncounter(?e) AND hasPatient(?e, ?p)
AND Patient(?p) AND primaryCode(?r, ?c)
AND RespiratoryCode(?c)
-> respiratoryReviewFor(?r, ?p)
```

The rule joins two branches of the record: its encounter-to-person path and its primary-code classification. The notebook begins with zero `respiratoryReviewFor` assertions, runs the supported HermiT rule path, and compares the resulting 20 record/person pairs with the source rows. It uses actual encounter and patient identifiers; it does not add a clinical outcome.

Use it, or choose another mechanism?

Requirement	Prefer	Reason
A subclass or equivalent-class definition	OWL	Standard ontology semantics and broader tool support
A plain relation chain	OWL property chain	Avoid a redundant rule when the axiom is sufficient
Supported positive join across variables	SWRL, after scope review	Can express a useful rule that needs shared variables
Required supplied code or range check	SHACL	Explicit graph validation with actionable reports
Count, latest row, or absence in a chosen dataset	SPARQL or SQL	Query and aggregation semantics are explicit
Time-dependent actions, API calls or task assignment	Python/application workflow	Controls state, retries and side effects

The first preference is the smallest semantic commitment that solves the problem. Do not choose SWRL simply because the rule syntax looks familiar. If a class restriction expresses the same meaning, the OWL axiom is often easier to exchange and profile-check.

What the lesson deliberately does not promise

Parsing a rule is not executing it. Executing one positive rule is not full SWRL conformance. A DL profile check on the core ontology does not certify a separate ontology containing rules. General SWRL adds expressivity beyond OWL DL; keep the rule policy, supported built-ins and named-domain behavior explicit.

The notebook uses no arithmetic built-ins and creates no new individuals in its rule head. Arithmetic support is engine-dependent. If a future rule needs a comparison or calculation, test that exact built-in with the pinned engine and runtime. Use SPARQL or Python when the transformation is straightforward and does not need to be part of the logical model.

Rules add consequences. They do not automatically retract a stored consequence after an input is removed. Rebuild the isolated rule ontology from the accepted source for this small dataset; larger deployments need reviewed truth-maintenance behavior. Missing input does not imply the opposite conclusion, and a rule should not send messages, change access rights or initiate clinical action as a hidden side effect.

Validation and reasoning are separate gates

An enterprise data contract should make omissions and datatype errors actionable. SHACL validates the graph provided to it against declared shapes. This lab validates the explicit intake graph with inference set to none. Requiring one primary code and one source encounter identifier is a delivery rule. It says what the producer must supply even when open-world logic could allow more information elsewhere.

The stayDays range is grounded in the source cohort's inclusion criteria, not a universal clinical law. A historical system with a different cohort could need a different shape. Test shapes with deliberately altered copies of real records: remove an identifier, add a second code, or change a typed count to text. Keep the original CSV and accepted graph unchanged.

A serving shape may differ from an intake shape. If a consumer expects an inferred ReviewCandidate type, validate the explicit-plus-approved-derived graph. Name the target class and inference policy so the same data does not pass in one deployment and fail in another for an invisible reason. Validation reports are RDF and can be cataloged with the run's provenance and model version.

An ordered release gate

Parse the RDF. Validate the explicit input contract. Assemble the reviewed logical imports closure. Check the chosen OWL profile. Check consistency and named-class satisfiability. Classify the supported query surface. Validate the serving graph. Reconcile competency-question answer identities. Publish a versioned product only after these gates succeed.

The order is practical rather than a universal standard. For example, some serving shapes intentionally target inferred classes and must run after classification. What matters is that the graph and inference policy at each gate are explicit. Running SHACL with undocumented RDFS inference can produce different targets from validating the raw graph.

Situation	OWL question	SHACL question
Missing documents edge	Can a model supply an unnamed filler?	Did the producer supply the required edge?
Two primary-code IRIs	Can they denote the same individual?	Does this graph have more than one value?
Patient and process type collision	Is the combined model inconsistent?	Does a chosen local shape also flag it?
Missing English preferred label	May another label exist elsewhere?	Does the vocabulary meet this publication rule?

The distinction is not that logic can never find errors. OWL can detect inconsistency and unsatisfiable classes. The distinction is that a data-delivery violation and a logical contradiction are different questions. A system often needs both answers.

Make failures teach

A useful negative test names the intended failure, changes one premise, and checks the resulting report. Do not only assert that a boolean is false; inspect the focus node, path and constraint component so a different accidental failure does not masquerade as the intended one. For deployment, route failures to an owner and preserve a reproducible input snapshot.

The source stay duration of 1–14 days is an intake-contract range because of the source selection policy. It should not become a universal ontology claim that no hospital encounter can last longer. This is a recurring pattern: a producer's finite scope belongs in a contract, not necessarily in the world's ontology.

SQL, mappings and reconciliation

The relational table is useful evidence, not an obstacle to semantic modeling. Start with the table's grain and candidate keys. Here the `encounter_id` identifies one row. The person and encounter are modeled separately, and counts are cast only after reading the source codes as strings. A type-2 slowly changing dimension row would need its own version identity as well as a link to the enduring entity.

SQL constraints and OWL axioms have different roles. A database unique constraint rejects duplicate keys; OWL functionality can imply that two fillers denote the same individual. SQL NULL and absent RDF values are not interchangeable truth values. The adapter must choose what a missing source token means. Aggregation counts solution rows in a selected graph; logical entailment decides whether a proposition follows.

Use reconciliation to discover hidden modeling changes. Compare the same population, filters, time window, units and code versions. A SQL count on source rows and a SPARQL count after a one-to-many label join can disagree even when neither parser reports an error. R2RML gives a standard mapping vocabulary; the supplied mapping file illustrates the subject template, logical table and predicate-object maps. The Python adapter is the executed implementation, not an R2RML processor.

A comparison with the same grain

```
SELECT diag_1, COUNT(*) AS encounters
FROM encounters
GROUP BY diag_1;
```

```
PREFIX ex: <https://example.org/health/ontology/>
SELECT ?code (COUNT(DISTINCT ?record) AS ?encounters)
WHERE {
  ?record a ex:EncounterRecord ; ex:primaryCode ?code .
}
GROUP BY ?code
```

The notebook converts the last code-IRI segment back to its source token before comparing the dictionaries. Both questions count the selected source records and produce three groups of 20. This equality is a contract test for this mapping; it is not a proof that every possible SQL and SPARQL query will be equivalent.

Historical rows and enduring entities

Suppose a warehouse later adds a slowly changing patient dimension. A source encounter refers to the patient, and the dimension record may describe that patient during a validity period. Keep the dimension record identity separate from the person identity. Preserve its validity interval and source provenance if those are supplied. Do not reuse an old row IRI to mean a different version without a declared update policy.

Avoid overmapping. The column `num_lab_procedures` gives a count, not the identities of procedures. The adapter cannot create 42 named laboratory events from the number 42 without adding unsupported information. The A1C category can be an information assertion; it cannot by itself establish an observation time, device or exact numeric result.

Materialized and virtual graphs

A materialized graph gives reproducible snapshots and local reasoning inputs at the cost of storage and refresh management. A virtual graph over relational data can reduce copying and preserve database operations, but its supported mappings, query rewriting and entailment regime need evaluation. Benchmark the queries that matter, including joins and filters that cross mappings. Do not assume an OWL QL label makes an arbitrary warehouse query rewrite efficient or complete.

The supplied R2RML file documents part of the mapping in a standard vocabulary. The runnable adapter is Python and RDFLib. This is an explicit implementation boundary, not a hidden claim that an R2RML processor was tested.

SPARQL from inspection to reliable answers

SPARQL should be learned as a sequence of answer-shaping operations. Start with a small graph and inspect the subject, predicates and object terms. Add a join and predict the number of solutions. Add optional information and inspect unbound values. Add aggregation only after the grain is stable. Finally add graph scope and a declared inference layer.

The ten SPARQL notebooks cover the original course's 19 topics through a revised topic crosswalk. Selected patterns from the additional SPARQL course strengthen debugging, multiset semantics, blank nodes, updates and portability. The running dataset is the same clinical subset used throughout the enterprise course.

The portable core

SELECT returns bindings; ASK reports whether a pattern has a solution; CONSTRUCT builds triples from a template; DESCRIBE returns an implementation-defined description. Use explicit CONSTRUCT patterns when a downstream consumer needs a stable shape. BIND assigns an expression, VALUES supplies a finite set of bindings, and COALESCE selects the first expression that evaluates without error.

OPTIONAL, UNION, MINUS and NOT EXISTS alter which solutions survive. Their placement matters. A FILTER outside OPTIONAL can discard records with missing values. MINUS with no shared variables leaves the left solutions untouched. Graph-level absence is not an OWL proof of nonmembership.

Count entities, not accidental join rows

```
PREFIX ex: <https://example.org/health/ontology/>
SELECT (COUNT(*) AS ?encounters)
      (COUNT(?alc) AS ?with_result)
WHERE {
  ?record a ex:EncounterRecord .
  OPTIONAL { ?record ex:alcCategory ?alc }
}
```

This returns 60 encounter solution rows and 18 bound A1C categories. Adding a two-valued VALUES dimension doubles solution rows to 120 while the distinct record count remains 60. The notebook makes this difference visible before introducing production-style joins.

Paths and subqueries

A path computes reachability over its graph. It can replace a fixed-depth hierarchy join when depths vary, but it does not supply OWL DL classification. Subqueries can preaggregate one branch, select a top group, or isolate a join grain. Define tie handling and deterministic ordering before presenting a result as “the first” or “the top.”

For pagination use an ordering with a stable tie-breaker. OFFSET over changing data can skip or repeat rows; cursor-style pagination over a stable ordered key is often easier to operationalize. A code string sorted lexically may have a different order from a numeric source identifier.

Fix the original course's portability issues

The revised sequence uses BIND instead of nonstandard LET, removes IFNULL from the portable function list, and expresses language filtering with LANG or LANGMATCHES rather than attaching a language tag to a variable. It separates SELECT/ASK JSON results from graph responses, adds HTTP timeouts, raises transport failures, and makes updates explicit. Ordinary parsing tests do not establish endpoint behavior; the optional HTTP integration is documented separately.

SPARQL and OWL DL: a precise query contract

The SPARQL 1.1 entailment-regimes recommendation defines conditions for OWL Direct Semantics evaluation. A normal RDFLib graph or an unconfigured Fuseki dataset does not acquire those semantics merely because it contains OWL triples. SPARQL-DL is also a separate query-language and engine approach, not a synonym for every SPARQL query over an ontology.

Our tested route is explicit: check the complete local imports closure, run HermiT, export entailed named class memberships for the named individuals, then query that finite view with ordinary SPARQL. This is sufficient for the stated review-class questions. Object-property entailments, negative assertions, arbitrary class-expression queries and unnamed witnesses are not exported by this adapter. A consumer requesting them needs a direct reasoner API or an endpoint with a verified entailment contract.

A useful query contract lists allowed graph patterns, ontology and data versions, import closure, consistency policy, reasoning engine and answer scope. Record expected positive and negative regression cases. If the ontology is inconsistent, stop serving its entailed answers. More triples do not make a contradictory model trustworthy.

Three distinct services

Service	What the query sees	Appropriate claim
Plain SPARQL on asserted RDF	Stored triples in a selected dataset	Graph pattern answers
SPARQL on an exported DL view	Stored facts plus selected entailed triples	Answers within the export's coverage
Verified OWL entailment endpoint or direct API	A defined logical query language and regime	Only the supported entailment guarantees

The course uses the second path for end-user queries and a direct reasoner API for selected semantic probes. The proof that aboutPatient follows from the property chain is tested through HermiT's entailment API. The class-membership export deliberately does not publish that inferred property. Consequently, a SPARQL query for aboutPatient on the exported graph returns nothing even though the specific proposition is entailed. This is a

useful coverage test, not a contradiction.

Why finite exports are useful

A finite named-class view is easy to cache, compare, authorize and explain. It gives predictable query behavior for a reviewed set of business questions. Its limitation is equally concrete: it cannot stand in for arbitrary DL reasoning. An existentially required encounter may remain unnamed; joins that need that witness are not answered merely by copying a few `rdf:type` triples.

Declare the ontology version, imports closure, data version, reasoning mode, consistency policy, exported predicates and named-answer policy. A consumer should be able to decide from that contract whether a proposed query is supported. If the product broadens its query surface, add semantic tests and performance evidence before changing the claim.

Open world and aggregate answers

COUNT over returned named candidates is a useful operational count. It is not the number of every entity that exists in all interpretations. Likewise, a missing triple in the asserted graph is not a negative assertion. Separate known positives, explicitly known negatives where modeled, and unresolved cases. Query-level NOT EXISTS can support a data-completeness workflow while remaining distinct from logical negation.

If an ontology is inconsistent, classical entailment does not provide meaningful discriminating answers. The serving policy here rejects publication. A reasoner reporting the contradiction is functioning correctly; the problem lies in the input model or facts and needs repair.

Provenance, evidence and graph boundaries

Use named graphs to make query scope visible. This dataset has asserted transformation output, vocabulary, selected derived memberships and metadata. The default graph is empty by design. A query without GRAPH is therefore not a shortcut to the union. Store products differ on default-union behavior, so this choice belongs in the data product's contract.

An answer should join a derived review classification back to the original source identifier and code. The explanation is a small proof sketch: the record has code 493; the adapter places that code in RespiratoryCode; the equivalent-class definition identifies a RespiratoryCodedRecord; the approved subclass axiom makes it a ReviewCandidate. This sketch is not an automatically minimal proof from the reasoner. Preserve the supporting rule and transform versions so another analyst can reproduce it.

Named graphs are useful packaging boundaries, not a security system. Authorization must apply to evidence as well as answers. A graph query that reveals a hidden source identifier through an inferred label still discloses information. This teaching product uses public data; an enterprise deployment must implement the relevant access policy at the service and export layers.

Capture lineage at the useful level

At dataset level, record the source snapshot and transformation run. At ontology level, record the model version and imports. At mapping level, record the reviewer decision and scope. At answer level, preserve the facts and rules needed to reproduce a conclusion. Full per-triple provenance can be expensive; choose a granularity that supports the actual audit questions.

The source manifest and dataset metadata are complementary. The manifest captures file checksums and selection details. The graph catalog lets consumers discover the distribution and its version. A PROV-O derivation link states a relationship but does not automatically prove that the transformation was correct. Reproduction and quality checks supply that operational evidence.

Separate categories of information

Keep source-derived assertions, terminology proposals, accepted mappings, selected reasoner consequences and editorial metadata in identifiable artifacts or graphs. This separation makes rollback and review easier. It does not require that every category have a different database; the important part is that queries and pipelines respect their stated roles.

The explanation in the capstone is a proof sketch written against known premises. It is not a minimal justification automatically extracted from HermiT. That distinction matters when a large ontology has several independent derivations. A production explanation service may need justification extraction, versioned rule traces or a carefully bounded proof template.

Security follows the evidence

Authorize the answer and its supporting source. Inference can reveal relationships not directly visible in one input graph, so a deployment must consider what can be derived across accessible graphs. A named graph alone cannot enforce those boundaries. Avoid sending private values to a federated endpoint or a generative model as an incidental consequence of a user query. The included public subset keeps the learning environment inspectable while the architecture makes these responsibilities visible.

Extension, versioning and retraction

A safe extension has a competency question, reviewed definitions, a profile check, a consistency check and answer regression. Extending an ontology is more than minting a class. A new subclass axiom can change every consumer that queries its superclass. An import update can add consequences outside the edited file. Version the ontology and its approved closure as one deployable dependency.

The cardiac extension broadens the review workflow from respiratory-coded records to cardiac-coded records as well. It adds one subclass axiom in a local extension file. The operational rule is a review policy, not a new medical inference. The expected review set changes from 20 to 40 and the original 20 remain. That is an intentional semantic change which requires an updated product contract.

OWL entailment is monotonic for a fixed language and growing premise set: adding axioms does not invalidate earlier logical consequences. Removing or correcting premises is different. A stored materialization can keep obsolete consequences unless it is retracted or rebuilt. This small course rebuilds from accepted source facts and the chosen model version; it does not pretend to implement incremental truth maintenance.

A change request worth reviewing

State the business question the change enables, the proposed term or axiom, its definition, the source evidence, expected old and new answers, and the consumers affected. Attach a counterexample that should remain excluded. A review can then assess meaning rather than only Turtle syntax.

Keep the base ontology separate from a local extension. Use new IRIs for new meanings; deprecate rather than silently redefine published terms. Maintain a version IRI and a reproducible import catalog. A patch label is not a semantic guarantee: even a one-line subclass change can alter many answers.

Compare answer identities

The cardiac extension is accepted in the exercise as a broader review policy. The test verifies the original 20 respiratory records remain and exactly the 20 cardiac records are added. Merely checking that the total became 40 would miss a regression that dropped five respiratory records and added five unrelated ones.

Rollback restores the earlier source, ontology, mappings and derived view as a coherent release. Rolling back only the ontology file while retaining the newer materialization produces a mixed state. The course rebuilds the finite view from scratch because the data are small. Larger systems may use incremental dependency tracking, but that mechanism needs its own correctness tests.

When a source fact disappears

Removing one supporting primaryCode assertion removes the corresponding review membership after rebuilding. Open-world logic does not prove the record is not a candidate; it only stops entailing the candidate type from the remaining premises. The intake shape separately rejects the missing required code. This single experiment connects source quality, entailment and operational release policy.

The semantic data product and its operating model

A reusable semantic product needs a named consumer and owner, a discoverable entry, an access policy, freshness and quality expectations, a versioned schema, support procedures and a cost model. DCAT describes datasets, distributions and data services. PROV-O expresses derivation and processing lineage. Neither vocabulary supplies an authorization mechanism or guarantees the metadata is true.

The course creates a real catalog entry for the included CSV distribution. Its download locator is a local artifact URN, because there is no deployed public service. A production catalog would replace that locator with an actual accessible distribution URL and add the relevant DataService entry. Advertising a nonexistent endpoint as running would break the product contract.

Publish source and ontology checksums, transformation version, reasoner configuration, accepted mapping set, validation result and run time with each release. Distinguish source vintage from refresh cadence: regularly rebuilding a dataset from 1999–2008 does not make the clinical observations current. Product adoption also depends on documentation, training and meaningful feedback channels; metadata completeness alone does not establish use.

Ownership with decisions attached

Role	Decision responsibility
Product owner	Consumer priorities, outcome measures and rollout
Ontology steward	Definitions, axioms, imports and semantic compatibility
Terminology steward	Mapping scope, status and version alignment
Data steward	Source contract, quality and provenance
Platform owner	Deployment, access, runtime and recovery
Consumer representative	Task usefulness and understandable evidence

One person can perform several roles in a small pilot, but the decisions still need named accountability in an actual deployment. A term dispute should have an escalation path and a recorded resolution. A release should identify who approved the meaning change and who handles affected consumers.

Service expectations

Specify freshness, availability, latency and support in the context of the product's data and users. For a historical teaching dataset, freshness means that the build reflects the reviewed source snapshot and model version. It does not mean the underlying clinical events are current. A service promise that omits this distinction can mislead consumers even when the software works perfectly.

Estimate compute cost, storage, stewardship effort, training effort and support time. Measure cost per successful task when the pilot has real task observations. Internal reuse and reduced reconciliation work can be valuable even when there is no external data sale. Keep the baseline and attribution of the benefit explicit.

Discoverability is tested through use

Ask a new consumer to find the dataset, locate the code scheme, identify the source vintage, understand the inference scope and find support. If they cannot, improve the catalog entry and documentation. A technically complete metadata record that nobody can interpret is not an adopted product.

AI integration with a bounded reasoning service

A language model can help a learner draft competency questions, explain a query, suggest vocabulary candidates or summarize a reviewed proof sketch. It should not become an untracked replacement for the ontology's meaning or the source evidence. Keep the generation step outside the authoritative logical core.

A controlled question-to-answer path

Map a user's question to an approved intent or query template. Bind parameters as RDF terms. Enforce graph scope, read-only policy, query limits and timeouts. Execute against the declared source and entailment view. Validate the expected answer shape. Return source identifiers and a traceable explanation. Let the language model render that evidence in accessible language without adding new clinical assertions.

If the question is outside the query contract, ask for a narrower question or route it to an analyst. A fluent answer assembled from a partially supported query is still unsupported. The empty result from a limited export is especially dangerous: it can mean no stored match, unsupported entailment coverage, a hidden graph, an execution failure or no named answer. The service must distinguish these cases.

Treat retrieved text as data

Labels, descriptions and external documents can contain instructions or misleading claims. They should not control tool execution or access rules. A query assistant must use its application policy rather than instructions embedded in a retrieved description. Keep the allowlist of query operations and endpoints outside user-controlled metadata.

Evaluate the combined system

Create tasks with expected identifiers, required evidence and disallowed overclaims. Measure correct answers, unsupported assertions, source coverage, latency, reviewer effort and user task completion. Include questions designed to expose limits: an unapproved crosswalk, an absent observation time, a missing measurement and a query requiring an unnamed existential witness.

Adoption, mentoring and the capstone

The capstone brings the system together: parse source data; check the intake contract; build the logical import closure; validate the OWL profile; check consistency; classify named individuals; publish the chosen finite entailment view; reconcile expected answers; produce an explanation; and describe the product in a catalog. A failure at a gate should stop publication, not become an empty success result.

The technical scorecard checks source integrity, answer identity, required evidence, profile compliance, consistency and reproducible rebuilding. The adoption scorecard observes representative users completing realistic tasks. Ask them to locate a record, explain its review status, identify the original diagnosis token, and recognize an unapproved mapping. Record assistance, elapsed time and errors. Use feedback to change labels, evidence displays or training, then repeat the same task protocol.

Calculate task completion as successful tasks divided by attempted tasks. Track answer correctness separately, and report review time and operating cost with their denominators. Weekly active or returning users must be divided by the eligible user group, not every account in the organization. Do not publish adoption percentages until a pilot actually runs. A blank measurement field is more honest and more useful than an unmeasured result.

Your release review has four deliverables: working inference; explanation with source evidence; catalog entry and product contract; and a pilot scorecard covering correctness, task completion, review time, adoption and cost. A reviewer should be able to rerun the notebooks and explain one answer to a business stakeholder without reading the implementation.

A peer review rubric

Score each dimension from 0 to 2: 0 means missing or unsupported, 1 means partly complete, and 2 means reproducible and clearly explained. Assess source integrity, identity modeling, taxonomy quality, mapping evidence, BFO bridges, DL compliance, validation, query scope, explanation and user-task readiness. Require correction of semantic errors before averaging the remaining scores.

The teach-back exercise has two audiences. An engineer should be able to reproduce the graph and reasoner result. A business stakeholder should be able to explain what action the

result supports and what it does not establish. If either explanation fails, revisit the vocabulary and evidence design.

A pilot you can actually run

Recruit representatives of the intended consumer roles. Give each participant the same small set of tasks and the same source context. Observe completion and assistance without changing the task halfway through. Record errors separately from time. Interview people who stop using the product; their reasons can reveal missing trust, confusing labels or a workflow mismatch.

Use the measurements to choose an intervention: clearer evidence, a glossary change, a smaller query surface, more training, or a different delivery mechanism. Repeat the task protocol after the change. A high usage count caused by repeated failed attempts should not be celebrated as adoption.

Finish with four artifacts

Deliver a working inference, an explanation with evidence, a cataloged product with ownership and a pilot scorecard. Include the exact source and model versions, the tested query surface and known limits. Another learner should be able to extract the archive, run the cells, reproduce the answer identities and understand why pending mappings are excluded.

The core outcome is durable: learners will measure whether people can use the product, alongside checking whether its answers are correct.

Reference cards and troubleshooting

Modeling reference card

If you need to express...	Start with...
A preferred business term and synonyms	SKOS labels and scope notes
A cross-scheme terminology relationship	A reviewed SKOS mapping
Class inclusion	rdfs:subClassOf
Necessary and sufficient classification	owl:equivalentClass with a reviewed expression
Identity of two individuals	owl:sameAs only with identity evidence
A required supplied field	A SHACL property shape
An inferred answer with source support	A declared reasoner query and provenance
A discovery record	DCAT dataset/distribution metadata
Who can see the evidence	Application and platform authorization

Common symptoms

Symptom	First check	Next action
Empty SPARQL result	Graph scope and exact IRIs	Compare asserted versus derived coverage
Too many rows	Join cardinality	Inspect intermediate bindings; aggregate at the intended grain
Missing inherited type	Reasoning configuration	Run the appropriate reasoner or inspect the explicit hierarchy
Unexpected individual merge	sameAs, keys or functionality	Review identity assumptions and explicit differences
SHACL fails but OWL is consistent	Required explicit values	Decide whether the failure is a delivery contract issue
Ontology inconsistent	Recently added facts, bridges and imports	Isolate the conflicting module; do not publish entailed answers
Profile check fails	Global property restrictions and datatypes	Refactor the axiom or choose an appropriate profile
Results survive a deleted premise	Stale materialization	Rebuild or retract the dependent view
Correct answer, low adoption	User task observation	Improve evidence, language or workflow

Setup reference card

If Java cannot be found, install a supported JDK and verify `java -version` in the same terminal used to launch JupyterLab. If the notebook uses the wrong Python, choose the registered Ontology Lab kernel. If a library import fails, install `requirements.txt` in that environment. If a source checksum differs, stop and review the source version rather than overwriting the expected checksum.

The verification environment cannot open Jupyter TCP or IPC kernel transports. Solution cells are therefore executed in fresh IPython processes, one notebook at a time, with outputs retained. The normal Jupyter execution script is included for learners to run locally. Cell behavior and semantic tests are verified; the browser interface, remote endpoints and a human pilot are not claimed as tested.

Vocabulary glossary

Term	Working meaning
Ontology	Formal account of selected entities, properties and relationships
Knowledge graph	Identified resources and statements expressed with shared vocabularies
Taxonomy	Hierarchical organization of concepts
Alignment	Reviewed relationships between terms or models
TBox	Class-level axioms and descriptions
ABox	Assertions about individuals
RBox	Property-level axioms, including inverses and chains
IRI	Identifier whose scope and identity policy must be governed
Entailment	Consequence true in every model satisfying the premises
Validation	Checking supplied graph content against a declared contract
Materialization	Storing a chosen set of inferred consequences
Provenance	Information about origins, responsibility and transformations
Profile	A restricted ontology language with defined syntactic conditions
Data product	Data and semantics with consumers, ownership and service expectations